

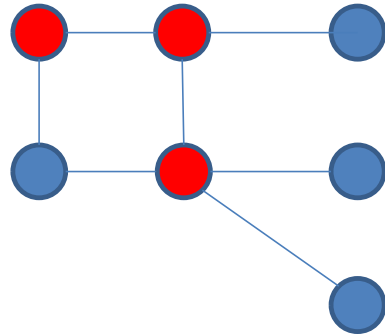
NP-Completeness

NP-complete problems

- Probably impossible to find efficient (that is polynomial time) algorithms to solve them
- However, they may be simple to formulate. Example: Vertex Cover
 - Given a graph G , a vertex cover for G is a subset C of the vertices such that, for every edge (v,w) of G , $v \in C$ or $w \in C$ (possibly both). Does G have a vertex cover of at most k vertices?

Vertex Cover, example

- Red nodes are a vertex cover



Decision Problems

- Computational problems which the intended output is either «yes» or «no»
- Examples
 - Given a string T and a string P , does P appear as a substring of T ?
 - Given two sets S and T , do S and T contain the same set of elements?
 - Given a graph G with integer weights on the edges, and an integer k . Does G have a minimum spanning tree of weight at most k ?

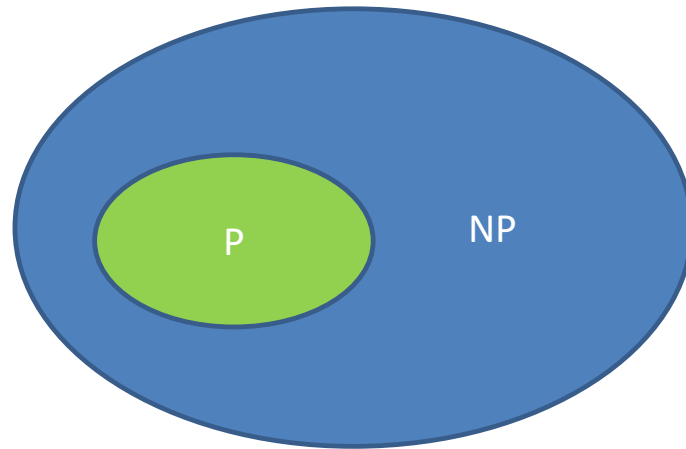
The Complexity Class P

- Set of all decision problems that we can solve in worst-case polynomial time (this is slightly inaccurate, but will do for now)
 - All the decision problems on the previous slide are in P
 - Have seen / will see polynomial time algorithms to solve them, and with small adjustments we can output yes/no

The Complexity Class NP

- Contains P but in addition we include decision problems where we can verify a «yes» answer in polynomial time
- Example, Vertex Cover
 - Have the graph G and a set of vertices C ?
 - Check that C contains at most k vertices
 - Run through all edges in G and verify that at least one of its endpoint are in C
- Hence Vertex Cover is in NP

Relation between P and NP



- We do not know if $P = NP$, but most scientists believe $P \neq NP$

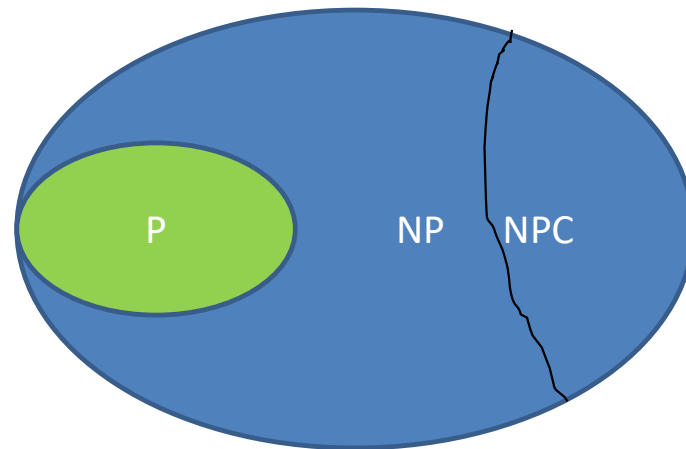
NP-Complete Problems

- Class of the most difficult problems in NP
- If one of them is solvable in polynomial time, all of them are
- To prove that a problem is NP-complete, we have to
 - Show that the problem is in NP
 - Show that a known NP-complete problem can be transformed (reduced) to our problem in polynomial time such that a solution to our problem corresponds to a solution to the known NP-complete problem

Cook-Levin theorem

- The Boolean satisfiability is NP-complete
- This was the first problem proved to be NP-complete and all later proofs of NP-completeness are founded directly or indirectly on this theorem
- If you will study for a masters degree and you want to learn more, INF235 (Complexity Theory) covers these in detail (the course is very theoretical)

Relation between P, NP and NPC



- Updated figure (with NPC-problems)

NP-complete problems

- Hamiltonian cycle
 - A Hamiltonian cycle in a graph is a simple cycle that includes all the vertices in the graph.
 - Does G have a Hamiltonian cycle?
- Clique
 - A clique in a graph G is a subset C of the vertices such that for each v and w in C , where $v \neq w$, (v,w) is an edge
 - Does G have a clique of size at least k ?

Approximation Algorithms

- One way to deal with optimization problems where the corresponding decision problem is NP-complete
- Such algorithms run much faster than an algorithm that finds the exact solution
- We will look at approximation algorithms for two problems
 - Vertex Cover
 - Travelling Salesperson

Terminology

- Have an instance of a problem where we have a set of feasible solutions, F .
- For each feasible solution, S , we have a cost function $c(S)$
- In a minimization problem, we want to find the solution S , such that
$$c(S) = OPT = \min\{C(T): T \in F\}$$
- We have a α -approximation algorithm if the algorithm always returns a feasible solution S such that $c(S) \leq \alpha OPT$

Value of α

- Some problems are so hard that we can not find approximation algorithm with $\alpha < \infty$
- For other problems, it is possible to find α –approximation algorithms with α as close to 1 as we want (but time used increases)

Vertex Cover

- Given a graph G , a vertex cover for G is a subset C of the vertices such that, for every edge (v,w) of G , $v \in C$ or $w \in C$ (possibly both).
- The optimization version of the problem is to find as small vertex cover for G as possible.

Approximation algorithm for VC

Algorithm VertexCoverApprox(G)

Input: a graph G

Output: A small vertex cover C for G

$C = \emptyset$

while G still has edges do

 select an edge $e = (v, w)$ of G

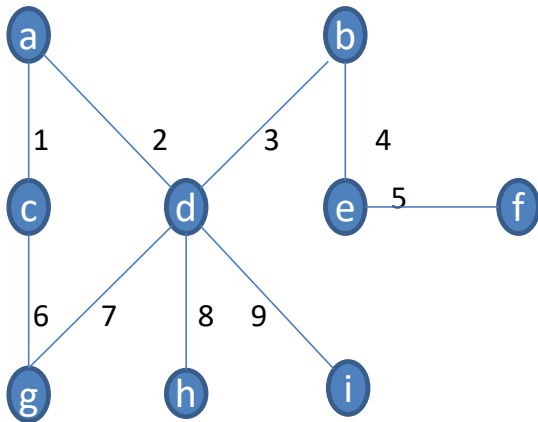
 add vertices v and w to C

for each edge f incident to v or w do

 remove f from G

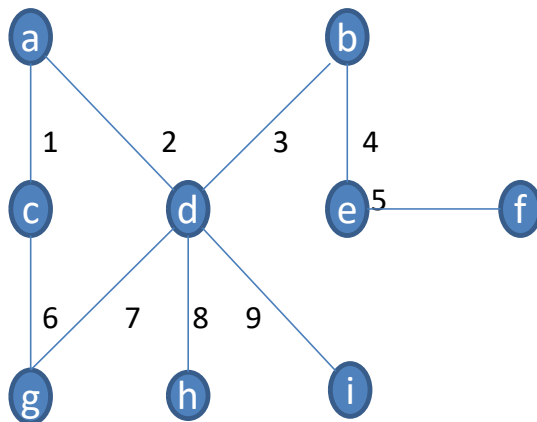
return C

Example (solution next slide)



- The number on the edges are in which order they are selected (unless they are removed earlier) by the algorithm. This is some random order.

Example



1: $C = \{a, c\}$, edge 2 and 6 removed

3: $C = \{a, c, b, d\}$, edge 4, 7, 8 and 9 removed

5: $C = \{a, c, b, d, e, f\}$

Optimal: $\{c, d, e\}$

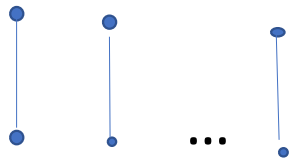
- The number on the edges are in which order they are selected (unless they are removed earlier) by the algorithm. This is some random order.

Theorem

- The algorithm for vertex cover is a 2-approximation algorithm
- Sketch of proof
 - For all the edges we select, we put both incident vertices in C
 - At least one vertex of each edge we selected has to be in the vertex cover (otherwise the edge is not covered)
 - Hence the vertex cover found by the algorithm can not contain more than twice the optimal (minimum) number of vertices

Notice

- The proof on the previous slide shows that α can not be greater than 2.
- To show it is at least 2, we can study the following example.



- Now the algorithm will pick all the vertices, but obviously half of them are sufficient

Travelling Salesperson (TSP)

- Have a weighted graph G . Want to find the minimum-weight cycle in G that visits all vertices in G .
- Consider a special case where the weight satisfy the **triangle inequality**, that is for any three edges (u,v) , (v,w) and (u,w) we have

$$c((u, v)) + c((v, w)) \geq c((u, w))$$

That is, the direct edge is not longer than going via another vertex

Euler tour

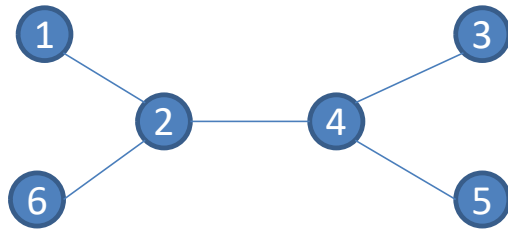
- An Euler tour starts in a vertex and visits all edges exactly once and ends in the start vertex
- A graph has an Euler tour if it is connected and all vertices have even degree
- There are several algorithms for finding an Euler tour efficiently
 - Can start in any vertex and traverse an unused edge unless the graph does fall apart

2-approximation algorithm for TSP

1. Construct a minimum spanning tree, M , of G
 - Use one of the algorithms we have seen (Kruskal, Prim-Jarnik, Boruvka)
2. Duplicate all edges in MST and let the M be the resulting graph. Then all vertices will have an even degree. Hence we can construct an Euler-tour traversal, E , of M
3. Construct a tour T from E by skipping already visited vertices (it is possible because all edges must be present in a connected graph that satisfies the triangle inequality)

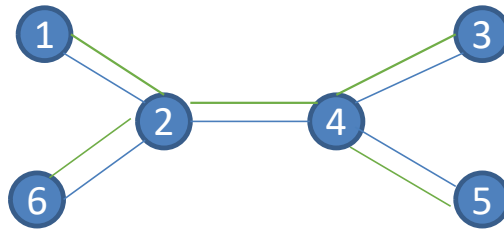
Example (1)

Blue edges = MST



Example (2)

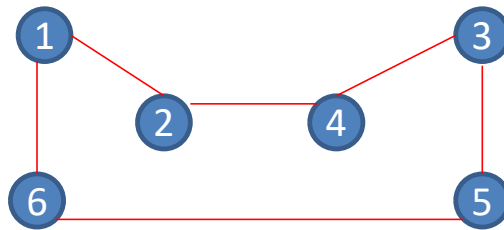
Green edges = duplicated edges



One Eulearing tour (travel edges in MST in twice): 1 2 4 3 4 5 4 2 6 2 1

Example (2)

Eulearing tour (travel edges in MST in twice): 1 2 4 3 4 5 4 2 6 2 1

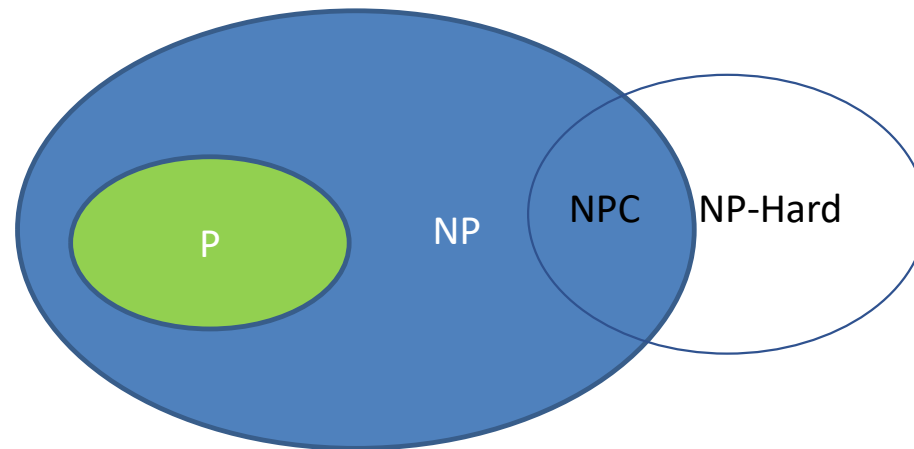


Travelling saleperson's tour (red edges): 1 2 4 3 5 6 1

Set cover problem

- Ground set of elements $E = \{e_1, e_2, \dots, e_n\}$
- Subsets of the elements $S_1, S_2, \dots, S_m$
- Each subset has a nonnegative weight $w_j \geq 0$
- Goal: Find a minimum weight collection of subsets that covers all of E . That is find a subset I of $\{1, 2, \dots, m\}$ that minimizes $\sum_{j \in I} w_j$ subject to $\bigcup_{j \in I} S_j = E$
- If $w_j = 1$ for each subset j , the problem is called the **unweighted** set cover problem

NP-hard



- Informally
 - Class of problems that are at least as hard as the hardest problems in NP

Some interesting facts

- The three following slides are for those that are very interested (and will not be part of the exam)

Some NP-hard problem are very hard

- Example
 - The halting problem for Python (there is an underscore in the link)
 - https://www.youtube.com/watch?v=r_GZ7ubU0M

NP-intermediate

- If $P \neq NP$, we know that there are problem in NP that are neither in P nor NP-complete (Ladner's theorem)

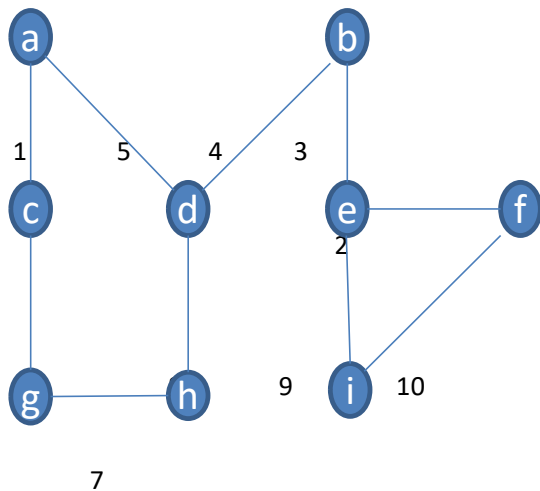
Quantum computing

- In principle quantum computers can solve some problems in NP-intermediate
- Quantum computers can find prime factors in polynomial time
- RSA cryptography depends on that if we have a number $n = p * q$, where p and q are (large) primes, then it is hard to find them just knowing n .
- Quantum-Safe cryptography tries to include NP-complete problems.

Problems

- Try yourself

Vertex Cover



- Find a vertex cover by the algorithm from lecture in the graph above. Try first, selecting edges in increasing order, then in decreasing order

Travelling Salesperson

	a	b	c	d	e	f
a		3	5	6	6	2
b			2	3	5	5
c				3	5	5
d					2	4
e						4
f						

- Find a Travelling Salesperson tour in the graph above (given by adjacency matrix) using the algorithm from lecture

Minimum spanning tree

Er pensum i tidlegare fag

